

Aula 08: Superescalar

ACH2055 – Organização e Arquitetura de Computadores II

Arquitetura de Computadores: uma abordagem quantitativa. Patterson e Hennessy, 5ª edição – Capítulo 3

Valdinei Freire da Silva

Escola de Artes, Ciências e Humanidades - USP

2025

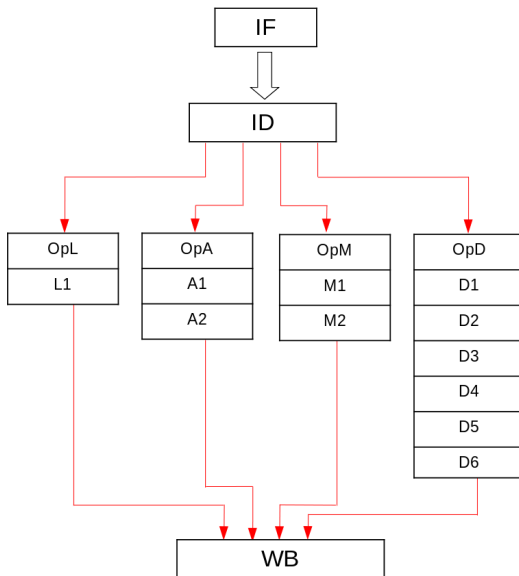
Agendamento com Scoreboard

Scoreboarding: instruções iniciam sempre em ordem, mas podem terminar fora de ordem quando há recurso suficiente e nenhuma dependência de dados

Estratégia para resolver conflitos:

- ▶ Estrutural - paraliza o despacho de instruções enquanto a unidade funcional não estiver disponível
- ▶ WAW - paraliza o despacho de instruções enquanto a operação em conflito ainda não escreveu no registrador
- ▶ RAW - segura a instrução na leitura de operandos até os operandos estarem disponíveis
- ▶ WAR - segura a instrução na execução enquanto os operandos não foram lidos

Agendamento com Scoreboard - Exemplo



Agendamento com Scoreboard - Exemplo

Sem Conflitos

- ▶ `div $r4, $r4, $r5`
- ▶ `or $r1, $r0, $r5`
- ▶ `add $r2, $r3, $r5`
- ▶ `lw $r7, 120($r6)`

Read after Write

- ▶ `div $r4, $r4, $r5`
- ▶ `lw $r1, 120($r4)`
- ▶ `add $r2, $r3, $r5`
- ▶ `or $r8, $r6, $r7`

Conflito Estrutural

- ▶ `div $r4, $r4, $r5`
- ▶ `add $r2, $r3, $r4`
- ▶ `add $r0, $r5, $r6`
- ▶ `or $r8, $r6, $r7`

WAW e WAR

- ▶ `div $r4, $r4, $r5`
- ▶ `or $r8, $r6, $r7`
- ▶ `add $r9, $r4, $r8`
- ▶ `lw $r8, 120($r6)`

Algoritmo de Tomasulo

Considere o código abaixo:

- ▶ I1: div \$r4, \$r4, \$r5
- ▶ I2: lw \$r1, 120(\$r4)
- ▶ I3: add \$r4, \$r3, \$r5
- ▶ I4: or \$r8, \$r6, \$r4

Note que há uma dependência WAR nas instruções I2 e I3, e uma dependência WAW nas instruções I1 e I3, ambas no registrador r4.

Se outro registrador for utilizado no lugar de r4 para as instruções I3 e I4, não existiriam mais os conflitos.

- ▶ I1: div \$r4, \$r4, \$r5
- ▶ I2: lw \$r1, 120(\$r4)
- ▶ I3: add \$X, \$r3, \$r5
- ▶ I4: or \$r8, \$r6, \$X

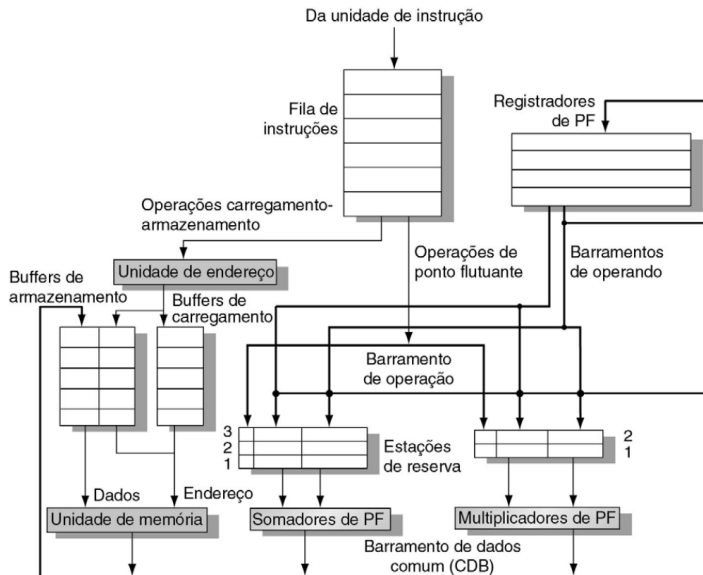
Algoritmo de Tomasulo

O algoritmo de Tomasulo permite renomear registradores dinamicamente para evitar conflitos WAW e WAR.

Estratégia:

- ▶ Estações de reservas para cada unidade funcional
- ▶ Uso de scoreboard para indicar onde localizar os operandos (registrador ou unidade funcionais)
- ▶ Uso de adiantamento para ler operandos na saída das unidades funcionais
- ▶ Quando escrita mais antiga estiver fora de ordem, cancelamento de escrita no registrador

Algoritmo de Tomasulo



Algoritmo de Tomasulo

Cada estação de reserva possui os seguintes campos:

- ▶ Busy. Indica que a estação de reserva está ocupada.
- ▶ Op. Operação a realizar na unidade.
- ▶ Qj, Qk. Unidades funcionais produzindo operandos-fonte (valor 0 indica que os operandos já estão disponíveis ou são desnecessário).
- ▶ Vj, Vk. Valor dos operandos-fontes.
- ▶ A. Informações sobre o cálculo do endereço de memória.

O banco de registradores possui um campo:

- ▶ Qi. O número da estação de reserva que contém a operação cujo resultado deve ser armazenado nesse registrador. Se estiver em branco, nenhuma instrução está calculando o valor para o registrador.

As informações entre Banco de Registradores, Estações de Reserva e Buffers são trocadas em um Barramento de Dados Comum.

Algoritmo de Tomasulo

Status da instrução	Esperar até	Ação ou manutenção
Despacho Operação de PF	Estação r vazia	<pre> if (RegisterStat[rs].Qi != 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi != 0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[r].Q ← r; </pre>
Load ou store	Buffer r vazio	<pre> if (RegisterStat[rs].Qi != 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes; </pre>
Apenas load		RegisterStat[rt].Qi ← r ;
Apenas store		<pre> if (RegisterStat[rt].Qi != 0) {RS[r].Qk ← RegisterStat[rs].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; </pre>
Execução Operação de PF	(RS[r].Qj = 0) e (RS[r].Qk = 0)	Calcular resultados: operandos estão em Vj e Vk
Load-store etapa 1	RS[r].Qj = 0 & r é o início da fila de store-load	RS[r].A ← RS[r].Vj + RS[r].A;
Load etapa 2	Load da etapa 1 concluído	Ler de Mem[RS[r].A]
Escrever resultado Operação de PF ou load	Execução completa em r & CDB disponível	<pre> ∀x(if (RegisterStat[x].Qi=r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); ∀x(if (RS[x].Qj=r) {RS[x].Vj ← result;RS[x].Qj ← 0}); ∀x(if (RS[x].Qk=r) {RS[x].Vk ← result;RS[x].Qk ← 0}); RS[r].Busy ← no; </pre>
Store	Execução completa em r & RS[r].Qk = 0	<pre> Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no; </pre>

Algoritmo de Tomasulo

Status da instrução									
Instrução	Despacho			Execução			Escrita de resultado		
L.D F6,32(R2)									
L.D F2,44(R3)									
MUL.D F0,F2,F4									
SUB.D F8,F2,F6									
DIV.D F10,F0,F6									
ADD.D F6,F8,F2									
Estações de reserva									
Nome	Busy	Op	Vj	Vk	Qj		Qk	A	
Load1									
Load2									
Add1									
Add2									
Add3									
Mult1									
Mult2									
Status dos registradores									
Campo	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi									

Algoritmo de Tomasulo

Instrução	Status da instrução		
	Despacho	Execução	Escrita de resultado
L.D F6,32(R2)	✓	✓	✓
L.D F2,44(R3)	✓	✓	
MUL.D F0,F2,F4	✓		
SUB.D F8,F2,F6	✓		
DIV.D F10,F0,F6	✓		
ADD.D F6,F8,F2	✓		

Estações de reserva							
Nome	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	não						
Load2	sim	Load					44 + Regs[R3]
Add1	sim	SUB		Mem[32 + Regs[R2]]	Load2		
Add2	sim	ADD			Add1	Load2	
Add3	não						
Mult1	sim	MUL		Regs[F4]	Load2		
Mult2	sim	DIV		Mem[32 + Regs[R2]]	Mult1		

Status dos registradores									
Campo	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1	Load2		Add2	Add1	Mult2			

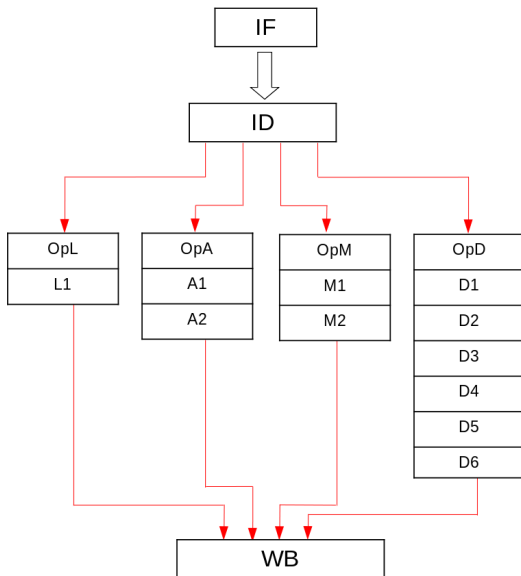
Algoritmo de Tomasulo

Instrução	Status da instrução		
	Despacho	Execução	Escrita de resultado
L.D F6,32(R2)	✓	✓	✓
L.D F2,44(R3)	✓	✓	✓
MUL.D F0,F2,F4	✓	✓	
SUB.D F8,F2,F6	✓	✓	✓
DIV.D F10,F0,F6	✓		
ADD.D F6,F8,F2	✓	✓	✓

Estações de reserva							
Nome	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	Não						
Load2	Não						
Add1	Não						
Add2	Não						
Add3	Não						
Mult1	Sim	MUL	Mem[44 + Regs[R3]]	Regs[F4]			
Mult2	Sim	DIV		Mem[32 + Regs[R2]]	Mult1		

Status dos registradores									
Campo	F0	F2	F4	F6	F8	F10	F12	...	F30
Qi	Mult1					Mult2			

Algoritmo de Tomasulo - Exemplo



Algoritmo de Tomasulo - Exemplo

Sem Conflitos

- ▶ `div $r4, $r4, $r5`
- ▶ `or $r1, $r0, $r5`
- ▶ `add $r2, $r3, $r5`
- ▶ `lw $r7, 120($r6)`

Read after Write

- ▶ `div $r4, $r4, $r5`
- ▶ `lw $r1, 120($r4)`
- ▶ `add $r2, $r3, $r5`
- ▶ `or $r8, $r6, $r7`

Conflito Estrutural

- ▶ `div $r4, $r4, $r5`
- ▶ `add $r2, $r3, $r4`
- ▶ `add $r0, $r5, $r6`
- ▶ `or $r8, $r6, $r7`

WAW e WAR

- ▶ `div $r4, $r4, $r5`
- ▶ `or $r8, $r6, $r7`
- ▶ `add $r9, $r4, $r8`
- ▶ `lw $r8, 120($r6)`